

Теория искусственных нейронных сетей

Каганов Ю.Т.

к.т.н., доцент каф. ИУ-9

**«Теоретическая информатика и компьютерные
технологии»**

ЛЕКЦИЯ № 11

**Нейронные сети
глубокого обучения**

Рекуррентные нейронные сети

Рекуррентные нейронные сети

Рекуррентные сети

- 1.1. Что такое рекуррентная сеть (RNN).
- 1.2..Области применения рекуррентных сетей.
- 1.3. Простейшие RNN.
- 1.4. Модель LSTM.
- 1.5. Модель GRU.

Что такое рекуррентные сети

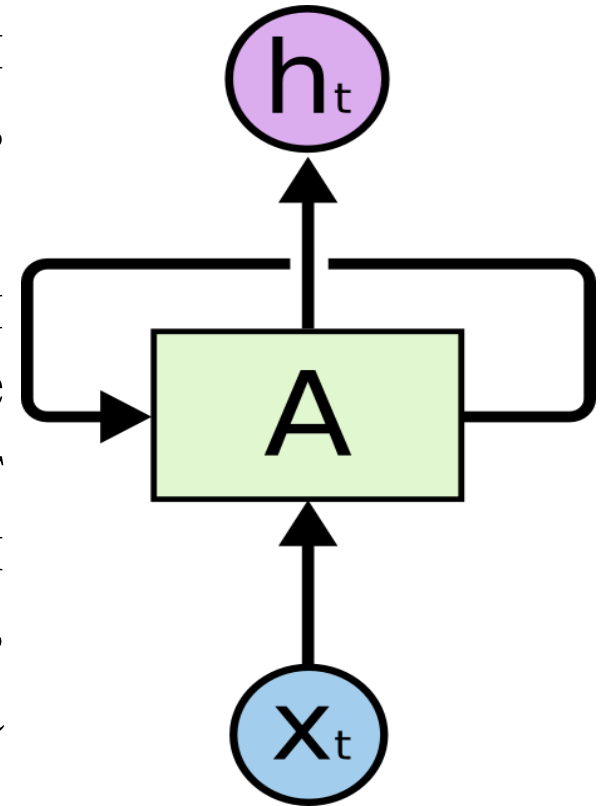
Рекуррентные нейронные сети РНС (*Recurrent neural network; RNN*,) — вид нейронных сетей, в которых имеется обратная связь. При этом под обратной связью подразумевается связь от логически более удалённого элемента к менее удалённому. Наличие обратных связей позволяет запоминать и воспроизводить целые последовательности реакций на один стимул. С точки зрения программирования в таких сетях появляется аналог циклического выполнения, а с точки зрения систем — такая сеть эквивалентна конечному автомату. Такие особенности потенциально предоставляют множество возможностей для моделирования биологических нейронных сетей.

Область применения РНС

- РНС – это класс нейронных сетей, в которых учитывается последовательный характер входных данных. На вход такой сети может подаваться текст, речь, временной ряд и т.п., в котором появление элемента в последовательности зависит от предшествующих элементов. РНС применяются для таких задач, как распознавание речи, языковое моделирование, машинный перевод, анализ эмоциональной окраски, подписывание изображений и многое другое.

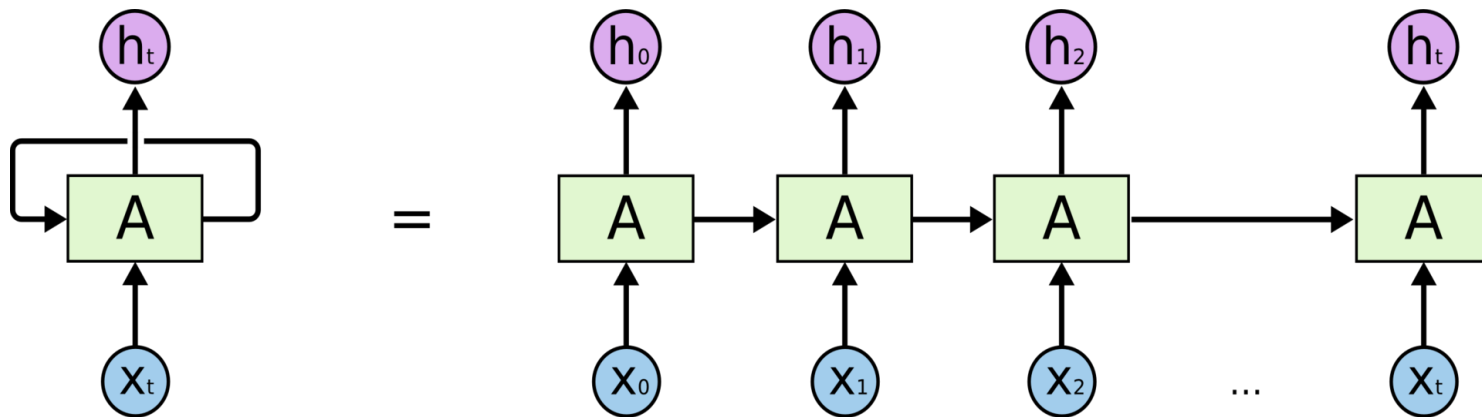
Рекуррентные нейронные сети

- RNN (РНС) — это сети, содержащие обратные связи и позволяющие сохранять информацию.
- Фрагмент нейронной сети A принимает входное значение x_t и возвращает значение h_t . Наличие обратной связи позволяет передавать информацию от одного шага сети к другому.



Рекуррентные нейронные сети

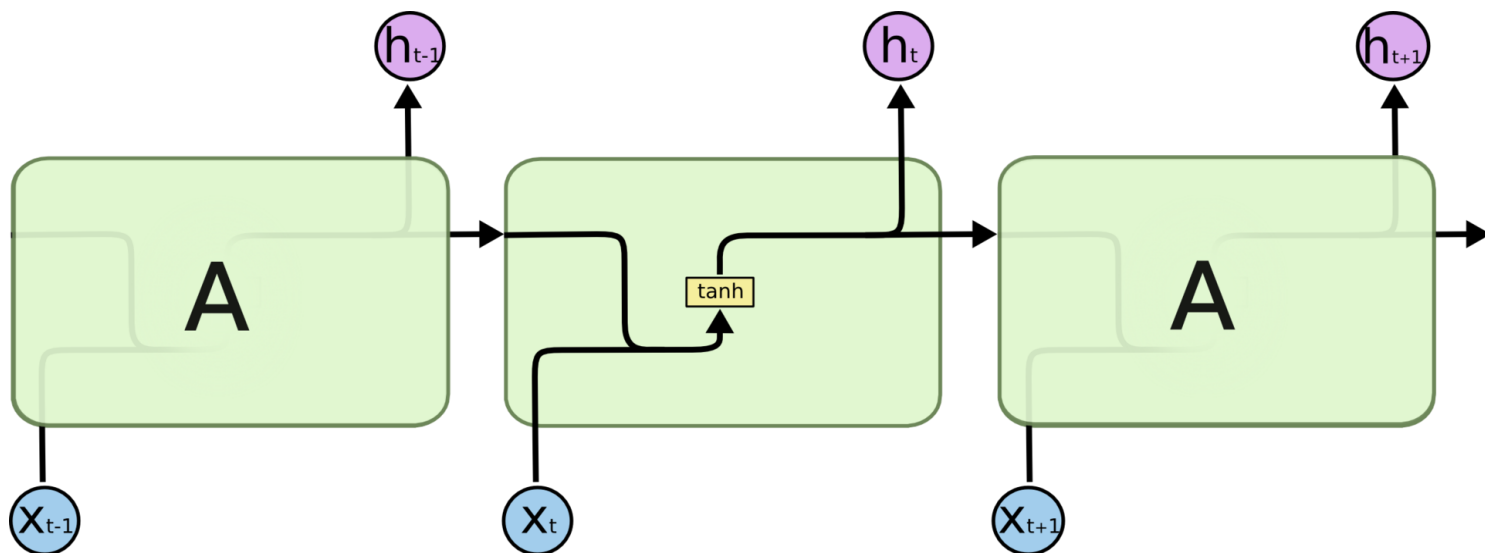
- Рекуррентная нейронная сеть в развертке



- Рекуррентную сеть можно рассматривать, как несколько копий одной и той же сети, каждая из которых передает информацию последующей копии.

Рекуррентные нейронные сети

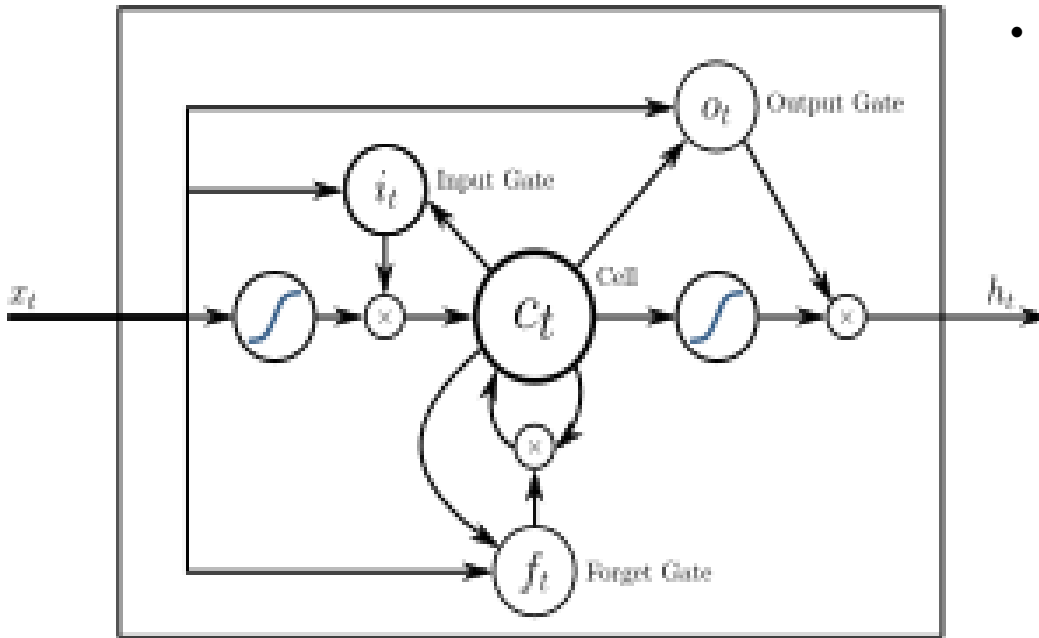
- Любая рекуррентная нейронная сеть имеет форму цепочки повторяющихся модулей нейронной сети. В обычной RNN структура одного такого модуля очень проста, например, он может представлять собой один слой с функцией активации $\tanh$ (гиперболический тангенс).



Сети LSTM

- Долгая краткосрочная память (Long short-term memory; LSTM) — особая разновидность архитектуры рекуррентных нейронных сетей, способная к обучению долговременным зависимостям. Они были представлены Зеппом Хохрайтером и Юргеном Шмидхубером в 1997 году, а затем усовершенствованы и популярно изложены в работах многих других исследователей. Они прекрасно решают целый ряд разнообразных задач и в настоящее время широко используются.

Модель LSTM



- *Long short-term memory*; *LSTM* — разновидность архитектуры рекуррентных нейронных сетей (RNN). Как и большинство рекуррентных нейронных сетей, LSTM-сеть является универсальной в том смысле, что при достаточном числе элементов сети она может выполнить любое вычисление, на которое способен обычный компьютер, для чего необходима соответствующая матрица весов, которая может рассматриваться как программа. В отличие от традиционных рекуррентных нейронных сетей, LSTM-сеть хорошо приспособлена к обучению на задачах классификации, обработки и прогнозирования временных рядов в случаях, когда важные события разделены временными лагами с неопределённой продолжительностью и границами.

Модель LSTM

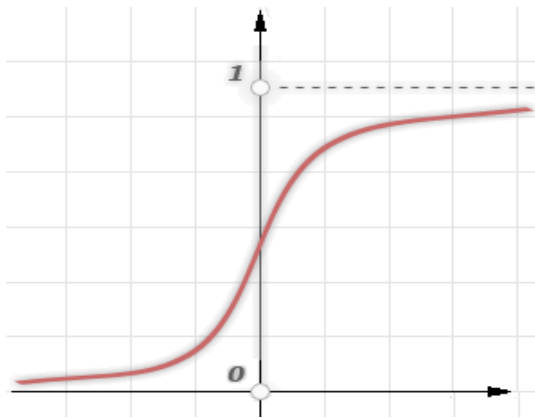
$$f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f),$$

$$i_t = \sigma_g(W_i x_t + U_i h_{t-1} + b_i),$$

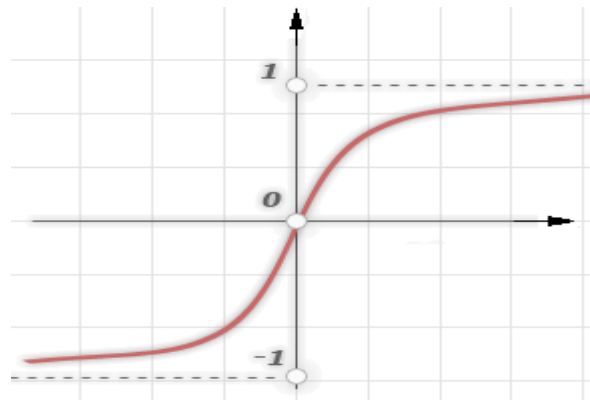
$$o_t = \sigma_g(W_o x_t + U_o h_{t-1} + b_o),$$

$$c_t = f_t \circ c_{t-1} + i_t \circ \sigma_c(W_c x_t + U_c h_{t-1} + b_c),$$

$$h_t = o_t \circ \sigma_h(c_t).$$



$$\sigma(s) = \frac{1}{1 + e^{-as}}$$



$$\sigma(s) = \frac{1 - e^{-as}}{1 + e^{-as}}$$

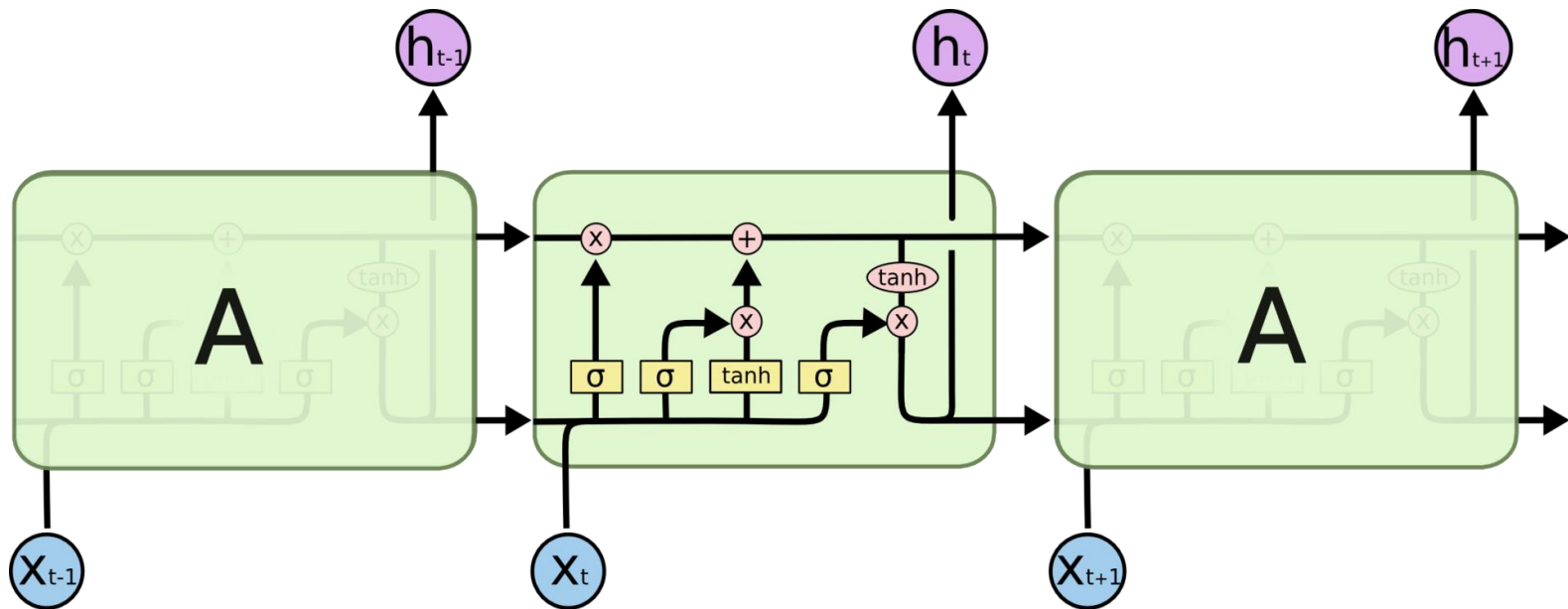
- Переменные
- x_t - входной вектор,
- h_t - выходной вектор,
- c_t - вектор состояния,
- W, U, b : - матрицы параметров и вектор порогов,
- f_t, i_t, o_t - векторы вентилей.
- Функции активации
- σ_g - на основе сигмоиды,
- σ_c - на основе гиперболического тангенса,
- σ_h - на основе гиперболического тангенса.

Модель LSTM

$$\left(\begin{array}{cccc|cccc} W_{xc} & & & & W_{hc} & & & \\ & W_{xi} & & & & W_{hi} & & \\ & & W_{xf} & & & & W_{hf} & \\ & & & W_{xo} & & & & W_{ho} \end{array} \right) * \begin{pmatrix} x_t \\ x_t \\ x_t \\ x_t \\ h_{t-1} \\ h_{t-1} \\ h_{t-1} \\ h_{t-1} \end{pmatrix} = \begin{pmatrix} W_{xc}x_t + W_{hc}h_{t-1} \\ W_{xi}x_t + W_{hi}h_{t-1} \\ W_{xf}x_t + W_{hf}h_{t-1} \\ W_{xo}x_t + W_{ho}h_{t-1} \end{pmatrix}$$

Структура LSTM

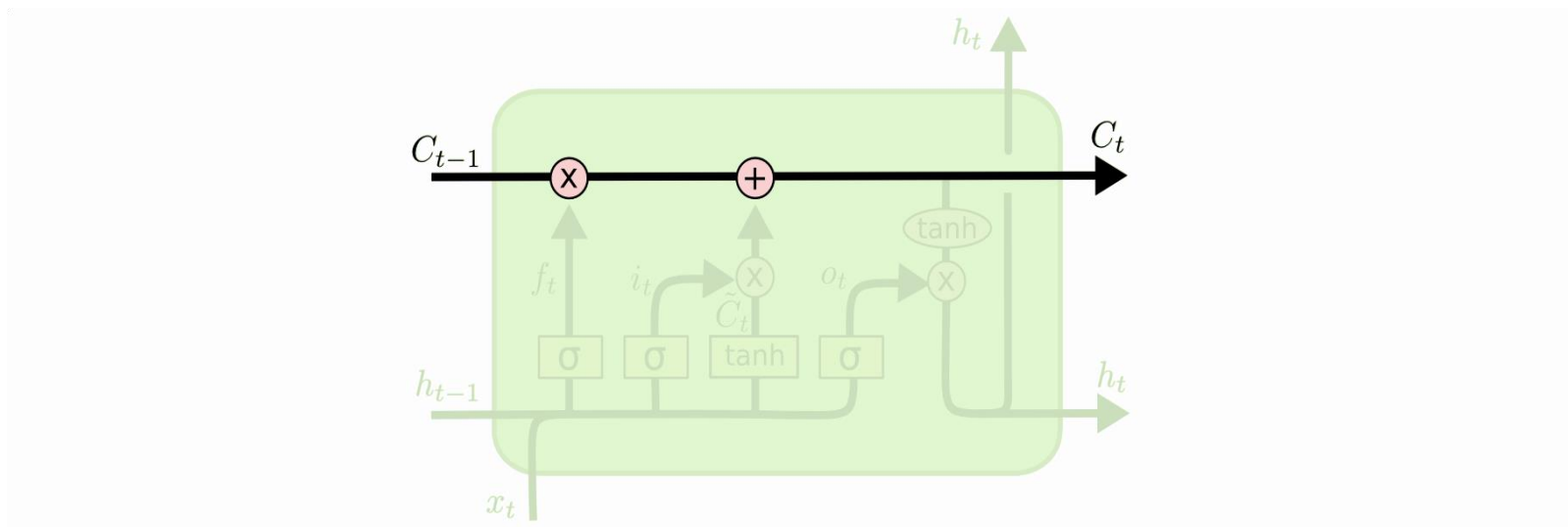
- Структура LSTM также напоминает цепочку, но модули выглядят иначе. Вместо одного слоя нейронной сети они содержат целых четыре, и эти слои взаимодействуют особым образом.



Структура LSTM

- На схеме каждая линия переносит целый вектор от выхода одного узла ко входу другого.
- Ключевой компонент LSTM — это состояние ячейки (cell state) — горизонтальная линия, проходящая по верхней части схемы.
- Состояние ячейки напоминает конвейерную ленту. Она проходит напрямую через всю цепочку, участвуя лишь в нескольких линейных преобразованиях. Информация может легко течь по ней, не подвергаясь изменениям.

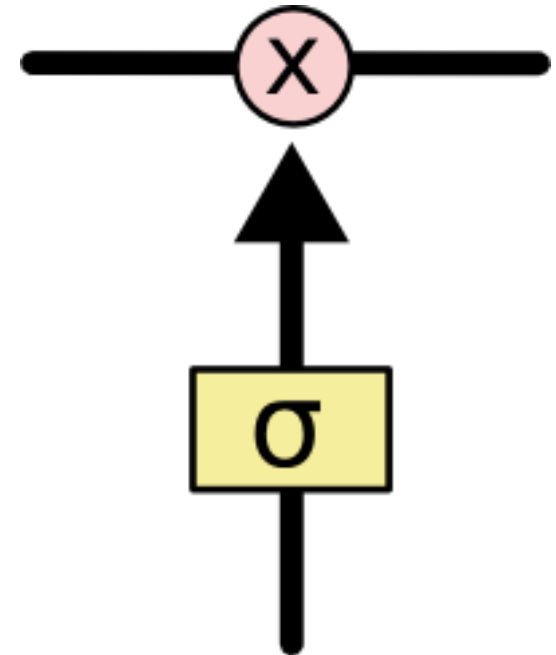
Структура LSTM



LSTM может удалять информацию из состояния ячейки; этот процесс регулируется структурами, называемыми фильтрами (gates).

Структура LSTM

- Фильтры позволяют пропускать информацию на основании некоторых условий. Они состоят из слоя сигмоидальной нейронной сети и операции поточечного умножения.
- Сигмоидальный слой возвращает числа от нуля до единицы, которые обозначают, какую долю каждого блока информации следует пропустить дальше по сети. Ноль в данном случае означает “не пропускать ничего”, единица — “пропустить все”.
- В LSTM три таких фильтра, позволяющих защищать и контролировать состояние ячейки.

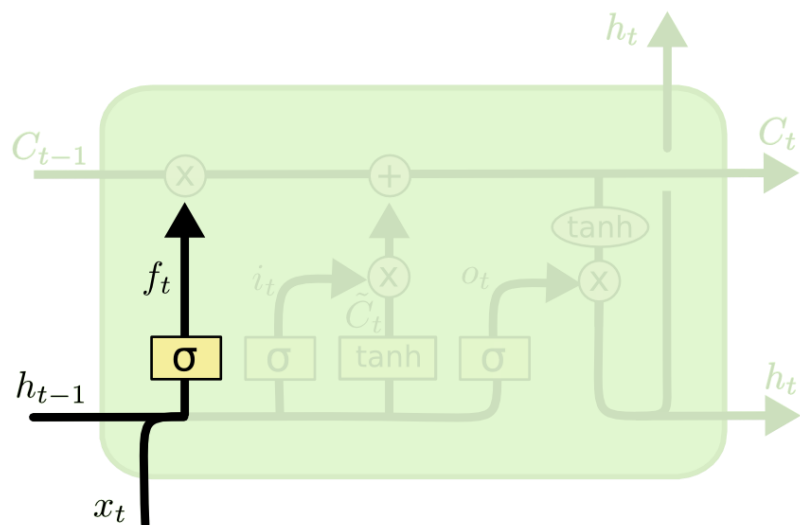


Структура LSTM

- Первый шаг в LSTM — определить, какую информацию можно выбросить из состояния ячейки. Это решение принимает сигмоидальный слой, называемый “слоем фильтра забывания” (forget gate layer). Он смотрит на h_{t-1} и x_t и возвращает число от 0 до 1 для каждого числа из состояния ячейки C_{t-1} . 1 означает “полностью сохранить”, а 0 — “полностью выбросить”.

Рассмотрим пример — языковую модель, предсказывающую следующее слово на основании всех предыдущих. В этом случае состояние ячейки должно сохранить существительное, чтобы затем использовать местоимения соответствующего рода. Когда мы видим новое существительное, мы можем забыть род старого.

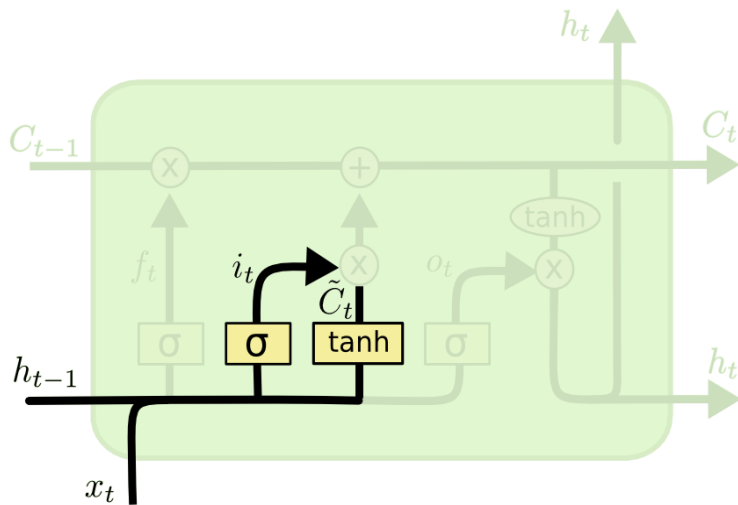
Структура LSTM



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

Структура LSTM

- Следующий шаг – решить, какая новая информация будет храниться в состоянии ячейки. Этот этап состоит из двух частей. Сначала сигмоидальный слой под названием “слой входного фильтра” (input layer gate) определяет, какие значения следует обновить. Затем tanh-слой строит вектор новых значений-кандидатов $\tilde{C}_t$, которые можно добавить в состояние ячейки.
В нашем примере с языковой моделью на этом шаге мы хотим добавить род нового существительного, заменив при этом старый.

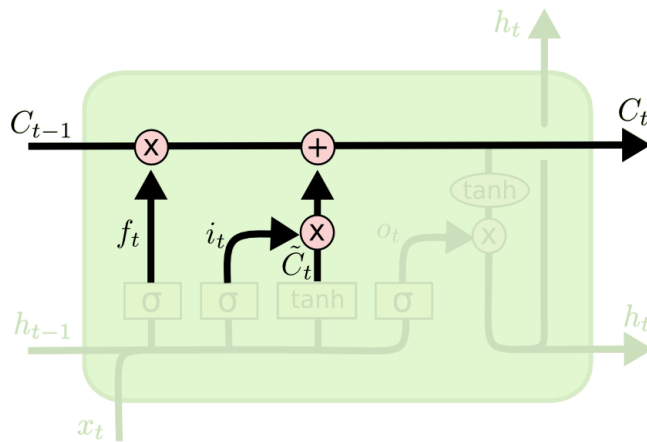


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Структура LSTM

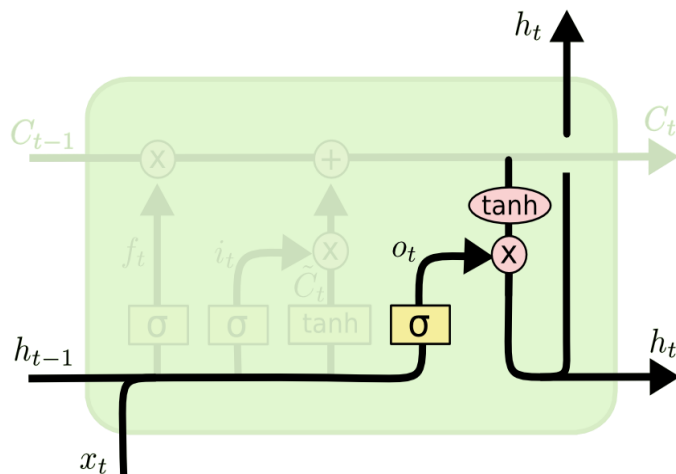
- Умножаем старое состояние на f_t , забывая то, что мы решили забыть. Затем прибавляем $i_t * \tilde{C}_t$. Это новые значения-кандидаты, умноженные на i_t — на сколько мы хотим обновить каждое из значений состояния. Для языковой модели это тот момент, когда мы выбрасываем информацию о роде старого существительного и добавляем новую информацию.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Структура LSTM

- Далее нужно решить, какую информацию мы хотим получать на выходе. Выходные данные будут основаны на нашем состоянии ячейки, к ним будут применены некоторые фильтры. Сначала мы применяем сигмоидальный слой, который решает, какую информацию из состояния ячейки мы будем выводить. Затем значения состояния ячейки проходят через tanh-слой, чтобы получить на выходе значения из диапазона от -1 до 1, и перемножаются с выходными значениями сигмоидального слоя, что позволяет выводить только требуемую информацию.



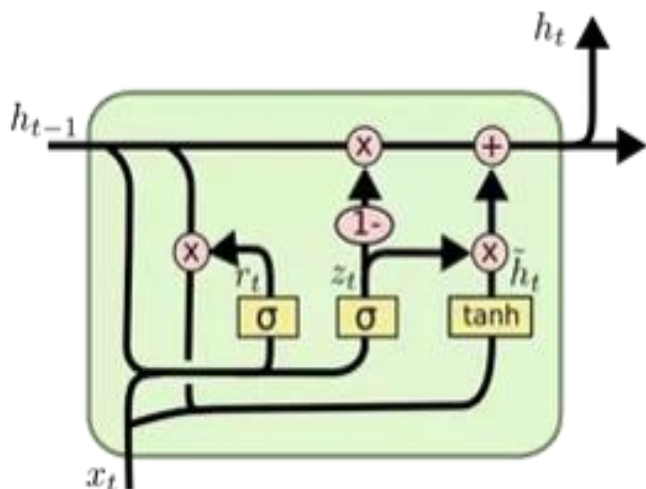
$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

GRU — управляемый рекуррентный блок

Gated Recurrent Unit

- В отличие от LSTM у GRU нет состояния ячейки и есть только два фильтра.



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

GRU — управляемый рекуррентный блок

- В управляемом рекуррентном блоке используются только фильтр обновления и фильтр сброса. Фильтр обновления отбирает информацию из предыдущих шагов, которая должна пойти дальше, а фильтр сброса решает, какую информацию из предыдущих шагов нужно отбросить. На рисунке Z_t — это фильтр обновления, он находит значения, которые должны пойти на следующие шаги с помощью сигмоидной функции. h_t — это фильтр сброса, который перемножает сцепленные входные данные этого шага и представления предыдущего с r_t и выдает значения из предыдущего шага, которые мы бы хотели бы убрать.

GRU — управляемый рекуррентный блок

- Не смотря на то, что GRU эффективнее в вычислительном плане из-за меньшего количества фильтров, он все равно стоит на втором месте после LSTM в плане исполнения. Поэтому GRU можно использовать тогда, когда нам нужно быстро обучить модель, а вычислительных мощностей не хватает.

Фреймворки для работы с глубокими сетями

- Определение графа вычислений
- Расчет графа вычислений
- Дифференцирование графа

Caffe


Chainer

DL4J
Deeplearning4j


KERAS

 Microsoft
CNTK

MatConvNet

MINERVA

mxnet


Purine


TensorFlow

theano

 torch

Типы фреймворков

- Фиксированные модули. Заранее определенные блоки комбинируются в граф вычислений и запускается. Максимальная скорость разработки, минимальная расширяемость. (**Caffe, Caffe2, DL4J, Keras**).
- Статический граф вычислений. Описывается сколь угодно сложный граф вычислений, потом он компилируется. После этого доступно прямое и обратное распространение. Появляется гибкость, но очень сложная отладка. (**Theano, TensorFlow, MXNet**).
- Динамический граф вычислений. Граф строится каждый раз при прямом проходе, чтобы потом иметь возможность сделать проход обратный. Максимальная гибкость, расширяемость, но проигрывает в скорости. (**Torch, PyTorch**).
- **Torch** — MATLAB-подобная библиотека для языка программирования Lua с открытым исходным кодом, предоставляет большое количество алгоритмов для глубинного обучения и научных расчётов. Ядро написано на Си, прикладная часть выполняется на LuaJIT, поддерживается распараллеливание вычислений средствами CUDA и OpenMP.

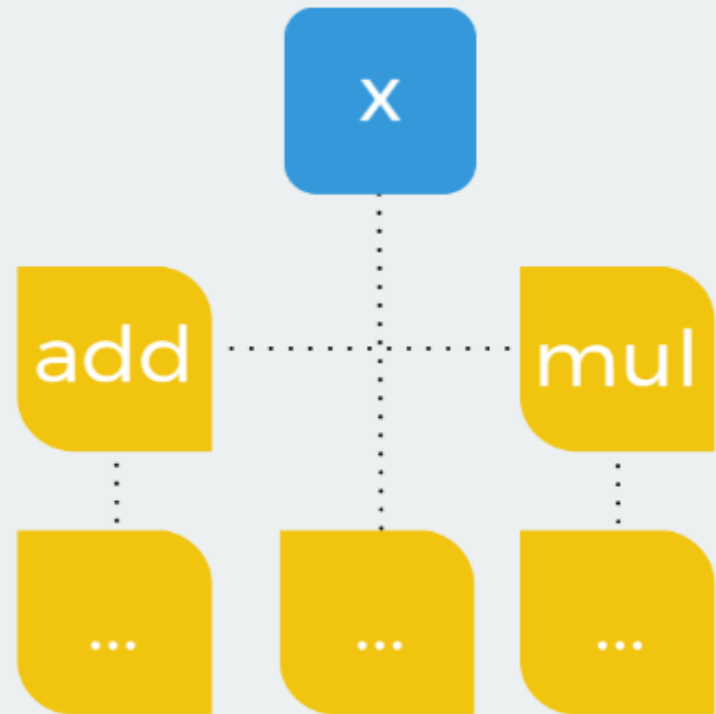
Типы фреймворков (2)



Caffe
Caffe2
CNTK
Kaldi
DL4J
Keras



Theano
TensorFlow
MXNet



Torch
PyTorch

Недостатки коннекционистского подхода

1. Отсутствие глубокого понимания устного и письменного текста.
2. Невозможность для современных искусственных нейронных сетей к внутренне мотивированным решениям.
3. Отсутствие способности к интуитивным решениям и сложным обобщениям (инсайт, сложные ассоциативные процессы запоминания и воспроизведения информации - поэзия).
4. Отсутствие способности распознавать и выделять глубокие причинно-следственные связи.

ЛИТЕРАТУРА

1. Хайкин С. Нейронные сети: полный курс. – М.: «Вильямс», 2006. – 1104 с.
2. Гудфеллоу Я. Бенджио И., Курвиль А. Глубокое обучение / пер с англ. А.А. Слинкин. – 2-е изд. – М.: ДМК Пресс, 2018. – 652 с.
3. Николенко С., Кадурын А., Архангельская Е. Глубокое обучение. – СПб.: Питер. 2018. – 480 с.
4. Чару Аггарвал Нейронные сети и глубокое обучение: учебный курс. Пер. с англ. – СПб.: ООО «Диалектика», 2020. – 752 с.
5. Лапань Максим Глубокое обучение с подкреплением. AlphaGo и другие технологии. – СПб.: Питер, 2020. – 496 с.
6. Рамсундар Б. TensorFlow для глубокого обучения: Пер. с англ. /Б. Рамсундар, Р.Б. Заде. – СПб.: БХВ-Петербург, 2020. – 256 с.
7. Эли Стивенс, Лука Антига, Томас Виман PyTorch. Освещаая глубокое обучение. – СПб.: Питер. 2022. – 576 с.